



**Università degli Studi dell'Insubria
Dipartimento di scienza e alta tecnologia
Corso di Laurea Magistrale in Fisica
Anno accademico 2025/2026**

Classificazione dei corpi celesti

Autore: Manganiello Loris

Matricola: 732609

Relatore: Professor Landoni Marco

Indice

1	Introduzione	2
1.1	Dati astronomici	2
1.2	AI, intelligenza artificiale	3
2	Presentazione dei dati	3
2.1	I parametri fotometrici UGRIZ	5
3	Pulizia dei dati	5
4	Algoritmo di ML	7
4.1	k-NN	8
4.1.1	Arricchimento del dataset	10
4.1.2	Questione del numero dei primi vicini (k)	10
5	Algoritmo di Beyes	12
6	Random Forest	13
6.1	Questione del numero di alberi	14
7	Confronto tra i tre modelli	15
8	Curva ROC	16
9	Validazione K-fold	17
10	Curva di apprendimento	17
11	Reti neurali	19
11.1	Introduzione alle Reti Neurali	19
11.2	Funzionamento delle Reti Neurali	19
11.2.1	Scelta della funzione di attivazione	20
11.2.2	Scelta della funzione di ottimizzazione	21
11.3	Algoritmo	21
11.4	Matrice di confusione	22
12	Conclusione	23

Sommario

L'obiettivo del seguente elaborato verte sulla classificazione di alcuni corpi celesti, sulla base di alcune delle loro proprietà fisiche note.

Verrà mosostrato il processo mediante il quale è possibile costruire un algoritmo di intelligenza artificiale specializzato nella classificazione.

In aggiunta verrà mostrato un confronto tra alcuni dei modelli di machine learning più noti, mettendo in luce i loro punti di forza e di debolezza, mostrando quale si adatta meglio al problema preso in esame.

1 Introduzione

Negli ultimi decenni l'essere umano ha compiuto enormi progressi nella conoscenza dell'Universo, grazie all'avanzamento delle tecnologie osservative e alla disponibilità di grandi survey astronomici. Tuttavia, a fronte di queste scoperte, emerge una nuova difficoltà: l'immagazzinazione, la gestione e l'analisi di quantità sempre più vaste di dati, che superano di gran lunga le capacità di un'analisi manuale tradizionale, cosa che poteva avvenire (non senza difficoltà) negli scorsi decenni. In questo contesto, l'intelligenza artificiale e le tecniche di machine learning si rivelano strumenti fondamentali, in grado di automatizzare i processi di classificazione e riconoscimento, permettendo così di estrarre informazioni importanti da dataset astronomici di dimensioni che altrimenti sarebbero ingestibili.

1.1 Dati astronomici

L'astronomia moderna si trova nell'era dei Big Data. I grandi survey astronomici, come lo Sloan Digital Sky Survey (SDSS) e il futuro Legacy Survey of Space and Time (LSST) dell'Osservatorio Vera Rubin, hanno rivoluzionato il modo di osservare il cosmo.

Questi progetti non si limitano a osservazioni mirate di singoli oggetti, ma hanno l'obiettivo di mappare porzioni estese di cielo in modo sistematico e uniforme. Grazie a telescopi dotati di camere fotometriche multibanda e di spettrografi ad alta risoluzione, i survey raccolgono ogni notte enormi quantità di dati che comprendono:

- Dati fotometrici: misure della luminosità degli oggetti in diverse bande (ad esempio u, g, r, i, z), utili per ricostruire la distribuzione spettrale dell'energia e distinguere tra differenti tipi di sorgenti;
- Dati spettroscopici: spettri dettagliati della radiazione emessa, che permettono di determinare la composizione chimica, la temperatura, la velocità radiale (tramite effetto Doppler) e il redshift cosmologico;
- Dati astrometrici: posizione precisa degli oggetti nel cielo e, in alcuni casi, misure di moto proprio e parallasse, fondamentali per stimarne la distanza.

La combinazione di queste informazioni consente di identificare e classificare miliardi di sorgenti celesti, dalle stelle della nostra Galassia, alle galassie lontane, fino ai quasar, nuclei galattici attivi estremamente luminosi.

Tuttavia, la quantità di dati prodotta da questi survey è immensa, lo SDSS, ad esempio, ha catalogato centinaia di milioni di oggetti, mentre l'LSST si prevede produrrà decine di terabyte di dati ogni notte. Questa mole di informazioni richiede metodi di analisi automatizzati, aprendo così la strada all'utilizzo di tecniche avanzate di machine learning per la classificazione e lo studio degli oggetti astronomici.

1.2 AI, intelligenza artificiale

L'Intelligenza Artificiale (AI) si è affermata come una delle tecnologie più innovative e trasversali in ambito scientifico e industriale. Con il termine AI si indica l'insieme di metodi e tecniche che consentono a un programma informatico di svolgere in autonomia compiti che, altrimenti, richiederebbero capacità umane, come il riconoscimento di pattern, la previsione di eventi o la presa di decisioni.

All'interno di questo campo si colloca il Machine Learning (ML), l'approccio che permette ai sistemi di imparare apprendendo dai dati che gli vengono forniti, per poi mettere in pratica ciò che hanno imparato.

A differenza della programmazione tradizionale, in cui un algoritmo segue regole fisse definite a priori, nel machine learning il modello viene addestrato su un insieme di dati e sviluppa la capacità di riconoscere relazioni e regolarità, migliorando le proprie prestazioni man mano che la mole di dati che gli vengono forniti aumenta.

Le tecniche di machine learning si suddividono principalmente in tre categorie:

- Apprendimento supervisionato, in cui il modello viene addestrato su dati etichettati ad esempio, immagini di galassie classificate come ellittiche o spirali (come se avesse le soluzioni per controllare la correttezza delle sue decisioni), così da apprendere come generalizzare e classificare nuovi casi;
- Apprendimento non supervisionato, che individua strutture nascoste o raggruppamenti all'interno dei dati senza che siano fornite etichette (quindi senza avere la possibilità di verificare le sue decisioni), ad esempio per scoprire nuove classi di oggetti astronomici;
- Apprendimento per rinforzo, in cui l'algoritmo impara tramite tentativi ed errori, ricevendo una "ricompensa" per ogni decisione corretta.

Una fase cruciale del machine learning è l'addestramento del modello, in cui i dati di input vengono utilizzati per ottimizzare i parametri interni dell'algoritmo, minimizzando una funzione di costo che misura l'errore commesso. Successivamente, il modello viene valutato su dati nuovi, detti dati di test, per verificarne la capacità di generalizzare e per verificare quanto correttamente l'algoritmo funziona.

Nell'ambito dell'astronomia, il machine learning rappresenta uno strumento di importanza crescente, poiché consente di affrontare sfide legate all'analisi di dataset estremamente ampi e complessi, migliorando l'efficienza e l'accuratezza dei processi di classificazione e scoperta scientifica.

2 Presentazione dei dati

In astronomia, la classificazione stellare è la classificazione delle stelle basata sulle loro caratteristiche spettrali. Lo schema di classificazione di galassie, quasar e stelle è uno dei più fondamentali in astronomia.

L'analisi presentata in questo elaborato è stata effettuata a partire dal file scaricato da *kaggle.com* chiamato "star_classification.csv", esso ha l'obiettivo di classificare stelle, galassie e quasar in base alle loro caratteristiche spettrali.

I dati consistono in 100.000 osservazioni dello spazio effettuate dall'SDSS (Sloan Digital Sky Survey). Ogni osservazione è descritta da 17 colonne di caratteristiche e 1 colonna di classe, che identifica se si tratta di una stella, di una galassia o di un quasar.

Una volta letti i dati presenti nel database mediante la funzione della libreria pandas `pd.read_csv("star_classification.csv")`, si presentano nel seguente modo:

	obj_ID	alpha	delta	u	g	r	i	z	run_ID	rerun_ID	cam_col	field_ID	spec_obj_ID	class	redshift	plate
0	1.237661e+18	135.689107	32.494632	23.87882	22.27530	20.39501	19.16573	18.79371	3606	301	2	79	6.543777e+18	GALAXY	0.634794	5812
1	1.237665e+18	144.826101	31.274185	24.77759	22.83188	22.58444	21.16812	21.61427	4518	301	5	119	1.176014e+19	GALAXY	0.779136	10445
2	1.237661e+18	142.188790	35.582444	25.26307	22.66389	20.60976	19.34857	18.94827	3606	301	2	120	5.152200e+18	GALAXY	0.644195	4576
3	1.237663e+18	338.741038	-0.402828	22.13682	23.77656	21.61162	20.50454	19.25010	4192	301	3	214	1.030107e+19	GALAXY	0.932346	9149
4	1.237680e+18	345.282593	21.183866	19.43718	17.58028	16.49747	15.97711	15.54461	8102	301	3	137	6.891865e+18	GALAXY	0.116123	6121
5	1.237680e+18	340.995121	20.589476	23.48827	23.33776	21.32195	20.25615	19.54544	8102	301	3	110	5.658977e+18	QSO	1.424659	5026
6	1.237679e+18	23.234926	11.418188	21.46973	21.17624	20.92829	20.60826	20.42573	7773	301	2	462	1.246262e+19	QSO	0.586455	11069
7	1.237679e+18	5.433176	12.065186	22.24979	22.02172	20.34126	19.48794	18.84999	7773	301	2	346	6.961443e+18	GALAXY	0.477009	6183
8	1.237661e+18	200.290475	47.199402	24.40286	22.35669	20.61032	19.46490	18.95852	3716	301	5	108	7.459285e+18	GALAXY	0.660012	6625
9	1.237671e+18	39.149691	28.102842	21.74669	20.03493	19.17553	18.81823	18.65422	5934	301	4	122	2.751763e+18	STAR	-0.000008	2444

Ove le colonne hanno il seguente significato:

- **obj_ID** = Identificatore dell'oggetto, il valore univoco che identifica l'oggetto nel catalogo delle immagini utilizzato dal CAS;
- **alpha** = Angolo di Right Ascension (all'epoca J2000);
- **delta** = Angolo di Declinazione (all'epoca J2000);
- **u** = Filtro ultravioletto nel sistema fotometrico;
- **g** = Filtro verde nel sistema fotometrico;
- **r** = Filtro rosso nel sistema fotometrico;
- **i** = Filtro vicino all'infrarosso nel sistema fotometrico;
- **z** = Filtro infrarosso nel sistema fotometrico;
- **run_ID** = Numero della scansione utilizzato per identificare la scansione specifica;
- **rerun_ID** = Numero di riesecuzione per specificare come l'immagine è stata elaborata;
- **cam_col** = Colonna della fotocamera per identificare la linea di scansione all'interno della run;
- **field_ID** = Numero del campo per identificare ciascun campo;
- **spec_obj_ID** = ID univoco usato per oggetti spettroscopici ottici (questo significa che due osservazioni diverse con lo stesso spec_obj_ID devono condividere la stessa classe di output);
- **class** = Classe dell'oggetto (galassia, stella o quasar);
- **redshift** = Valore dello spostamento verso il rosso basato sull'aumento della lunghezza d'onda;
- **plate** = ID della lastra, identifica ciascuna lastra nell'SDSS;
- **MJD** = Julian Date Modificata, utilizzata per indicare quando è stato acquisito un dato SDSS;
- **fiber_ID** = ID della fibra che identifica la fibra che ha catturato la luce sul piano focale in ciascuna osservazione.

2.1 I parametri fotometrici UGRIZ

Nell'ambito dell'astronomia osservativa, un ruolo fondamentale è svolto dai survey fotometrici, i quali raccolgono dati sulla radiazione elettromagnetica proveniente da stelle, galassie e altri oggetti celesti.

Tra i più noti e rilevanti si colloca appunto lo Sloan Digital Sky Survey, che ha introdotto un sistema standardizzato di cinque filtri fotometrici, denominati *u*, *g*, *r*, *i*, *z*, spesso indicati complessivamente come UGRIZ.

Ciascun filtro corrisponde a una specifica banda di lunghezze d'onda dello spettro elettromagnetico: il filtro *u* (ultravioletto) è centrato a circa 355 nm, il filtro *g* (blu-verde) a 468 nm, il filtro *r* (rosso) a 616 nm, il filtro *i* (vicino infrarosso) a 748 nm e, infine, il filtro *z* (infrarosso più profondo) a 893 nm.

Ogni osservazione fornisce quindi la magnitudine apparente di un oggetto in ciascuna di queste bande.

L'utilità di questo approccio risiede nella possibilità di costruire i cosiddetti indici di colore, ottenuti dalla differenza tra magnitudini misurate in due filtri differenti (ad esempio $u - g$ o $g - r$, etc...).

Tali indici sono strettamente correlati alle proprietà fisiche degli oggetti astronomici, infatti consentono di stimare la temperatura superficiale delle stelle, la loro metallicità, la classe spettrale e, in generale, di distinguere tra popolazioni stellari differenti.

Inoltre, gli stessi parametri *ugriz* si rivelano efficaci nella separazione tra stelle, galassie e quasar, costituendo quindi una base essenziale per compiti di classificazione automatica tramite algoritmi di machine learning.

In sintesi, i parametri fotometrici *ugriz* rappresentano una codifica standardizzata delle informazioni spettrali degli oggetti celesti, permettendo di trasformare osservazioni complesse in un insieme di dati numerici direttamente utilizzabili per analisi statistiche e computazionali su larga scala.

3 Pulizia dei dati

Dei dati mostrati in Paragrafo [2], quindi, non tutti sono utili al fine dell'analisi.

Quelli che verranno utilizzati sono solamente i dati relativi ai parametri *ugriz* già descritti in Paragrafo [2.1] e ovviamente i dati relativi alla **class**, ovvero quelli necessari per rendere l'algoritmo supervisionato.

Per questo motivo i restanti dati possono essere eliminati dal database mediante la funzione `data.drop()`.

Delle rimanenti colonne, è necessario accertarsi che i dati non siano incompleti.

Non è raro che alcuni valori siano infatti dei NaN, per ovviare a questo problema viene utilizzata la funzione `data.isnull().sum()`, la quale trova e conta le colonne con dati incompleti.

Nel caso questi fossero presenti, andranno prontamente eliminati dal database per non inquinare i dati correttamente inseriti.

Si può notare che da Eq. (3) che i dati presenti in **class** sono scritti il caratteri latini, non in numeri.

Questo può generare problemi all'algoritmo poichè esso non sa lavorare con le parole ma solamente con i numeri.

Per ovviare a questo problema è necessario sostituire le parole con numeri identificativi. Nell'analisi presentata, i dati sono stati così sostituiti:

- *Galassia* verrà sostituito con 0;
- *Stella* verrà sostituito con 1;
- *Quasar* verrà sostituito con 2.

Per la sostituzione delle stringhe è stata utilizzata la funzione `data[""].str.replace("", "")`

Siccome i caratteri latini precedenti (*GALAXY*, *STAR* e *QSO*) erano delle stringhe, anche ora che sono state sostituite da numeri mantengono la stessa classe.

Per la risoluzione di questo problema è sufficiente specificare la tipologia di dato da utilizzare, mediante la funzione `data[""]=data[""].astype('int')`.

Nell'analisi effettuata è stato scelto come tipo di dato `int`, ciò nonostante è giusto dire che sarebbe andato ugualmente bene anche la tipologia `float`, in quanto il programma sarebbe stato capace ugualmente di elaborare il dato.

Una volta terminato il data cleaning, il database si presenta in questa forma:

	u	g	r	i	z	class
0	23.87882	22.27530	20.39501	19.16573	18.79371	0
1	24.77759	22.83188	22.58444	21.16812	21.61427	0
2	25.26307	22.66389	20.60976	19.34857	18.94827	0
3	22.13682	23.77656	21.61162	20.50454	19.25010	0
4	19.43718	17.58028	16.49747	15.97711	15.54461	0

4 Algoritmo di ML

L'obiettivo principale del codice è la classificazione automatica dei corpi celesti in base ai loro parametri fotometrici, utilizzando tecniche di apprendimento supervisionato.

In questo approccio, il modello di machine learning viene addestrato su un insieme di dati per i quali la classe di appartenenza è già nota, in modo che possa imparare a riconoscere pattern e relazioni tra le variabili fotometriche e la classe corrispondente.

Una volta che i dati sono stati puliti, si può passare all'algoritmo in grado di fare ciò.

Una decisione molto importante è circa la tipologia di algoritmo da utilizzare.

Infatti nel campo del Machine Learning esistono numerosi algoritmi, ciascuno con caratteristiche, punti di forza e limiti propri.

Tra i più utilizzati si trovano, ad esempio, il k-Nearest Neighbors (k-NN), il Naive Bayes, il Random Forest e le reti neurali artificiali.

La scelta dell'algoritmo più adatto dipende dal tipo di problema, dalla quantità e qualità dei dati disponibili, nonché dagli obiettivi dell'analisi.

Alcuni modelli sono più interpretabili ma meno potenti in termini predittivi, altri invece garantiscono prestazioni più elevate a scapito della trasparenza e del tempo di addestramento.

Per questo motivo, nella pratica del Machine Learning è fondamentale confrontare diversi algoritmi, valutarne i risultati e selezionare quello che meglio bilancia accuratezza, complessità e capacità di generalizzazione.

Nell'analisi in esame sono stati utilizzati tre diversi tipi di algoritmo: *k-NN*, *Naive Bayes* e *Random Forest*.

Per la creazione di questi, è necessario l'importazione della libreria `sklearn`.

Nell'analisi si è preferito costruire un algoritmo supervisionato. Quindi è stato diviso il database in due parti:

- Training set (80%), utilizzato per addestrare i modelli;
- Test set (20%), utilizzato per valutarne le prestazioni su dati non ancora "disponibili" all'algoritmo.

In generale non esiste una divisione obbligatoria, le percentuali vanno decise a seconda dei casi.

Va però tenuto conto che nel caso in cui il *training set* sia troppo esigui, l'algoritmo non avrà abbastanza dati per potersi allenare, e quindi le sue prestazioni potrebbero non essere ottimali.

Nel caso invece in cui sia il *test set* ad essere troppo esiguo c'è il rischio che l'algoritmo funzioni bene ma la statistica su cui noi raccogliamo i dati non sia sufficiente, portandoci quindi a non conoscere la reale accuratezza dell'algoritmo.

4.1 k-NN

Il primo algoritmo ad essere preso in esame è il k-Nearest Neighbors (k-NN). Questo è un metodo di classificazione basato sulla distanza.

Il suo principio è il seguente: un nuovo punto da classificare viene assegnato alla classe più comune tra i suoi k primi vicini, ovvero i k punti del dataset di training che si trovano più vicini a esso nello spazio delle features.

La vicinanza tra punti viene normalmente calcolata tramite la distanza euclidea ($d = \sqrt{x^2 + y^2}$), ma possono essere utilizzate anche altre metriche a seconda del tipo di dati.

In pratica, il k-NN non costruisce un vero e proprio modello durante la fase di addestramento, ma memorizza i dati e li utilizza direttamente al momento della predizione: per questo motivo è definito un algoritmo di tipo lazy learner.

La scelta del parametro k è fondamentale: valori piccoli rendono il modello più sensibile al rumore (fenomeno di overfitting), mentre valori troppo grandi possono ridurre la capacità di distinguere tra classi diverse (underfitting).

Tale algoritmo può essere scritto nel seguente modo:

```
fe=data[['u','g','r','i','z']]
ta=data['class']

fe_train,fe_test,ta_train,ta_test=
train_test_split(fe,ta,test_size=0.2,random_state=26)

model=KNeighborsClassifier(n_neighbors=5)
model.fit(fe_train,ta_train)
pred=model.predict(fe_test) 1
```

Durante l'addestramento dei modelli di machine learning vengono spesso introdotti elementi di casualità, ad esempio nella suddivisione del dataset in training e test set.

Per garantire la riproducibilità dei risultati, è quindi necessario fissare un valore di random seed, che consente di controllare il generatore di numeri casuali e ottenere sempre gli stessi risultati a parità di condizioni iniziali.

Mostrando a schermo i risultati ottenuti, è stata presentata la seguente schermata:

¹Le linee di codice che sono state scritte separate, sono state scritte in tale modo unicamente al fine di mostrare che la linea centrale, sebbene si vada a capo, costituisce una linea sola.

Accuratezza: 0.8664				
Risultato di classificazione:				
	precision	recall	f1-score	support
0	0.91	0.94	0.92	11909
1	0.80	0.75	0.77	4371
2	0.81	0.77	0.79	3720
accuracy			0.87	20000
macro avg	0.84	0.82	0.83	20000
weighted avg	0.86	0.87	0.86	20000

Figura 1: Accuratezza K-nn

Nell'immagine in Figura [1] si possono vedere i vari valore di accuratezza che l'algoritmo presenta.

In ordine si nota:

- *Precision*: Percentuale di predizioni corrette tra quelle etichettate come appartenenti a quella classe. Indica quanto “sicuro” è il modello quando predice una certa classe.
- *Recall*: Percentuale di campioni effettivamente appartenenti a una classe che il modello riesce a riconoscere. Misura la “completezza” del riconoscimento.
- *F1-score*: Media armonica di precision e recall: rappresenta un equilibrio tra le due.
- *Support*: Numero di campioni reali appartenenti a quella classe nel dataset.

Ciascuno di questi parametri viene analizzato per ogni classe di corpi celesti, ricordando che la classe 0 è riferita alle *Galassie*, la classe 1 è riferita alle *Stelle*, mentre la classe 2 è riferita alle *Quasar*.

In questo caso il modello ha raggiunto un'accuratezza complessiva pari a $Acc. = 0.8664$, dimostrando una buona capacità di generalizzazione.

Analizzando il report di classificazione, si osserva che la classe *Galassie* viene riconosciuta con le migliori prestazioni ($F1\text{-score} = 0.92$), mentre le classi *Stelle* e *Quasar* presentano valori leggermente inferiori ($F1\text{-score}$ compresi tra 0.77 e 0.79).

Le metriche medie (macro e weighted average) confermano un bilanciamento complessivamente soddisfacente tra precisione e richiamo, indicando che il modello riesce a gestire in modo equilibrato le diverse classi pur con leggere variazioni nelle performance.

Il motivo per cui il modello classifica così bene la classe delle galassie invece che le altre due, potrebbe essere da ricercarsi nel fatto che quella è la classe più numerosa del database.

4.1.1 Arricchimento del dataset

Per migliorare la capacità discriminativa del modello, è possibile introdurre un database che non contiene solo le magnitudini per ciascun oggetto (*ugriz*), ma anche le differenze (dette colori) tra coppie di bande adiacenti (cioè $u - g$, $g - r$, $r - i$, $i - z$).

Questo passaggio è motivato dal fatto che le magnitudini singole possono essere fortemente influenzate da effetti sistematici quali la distanza, l'estinzione, la luminosità intrinseca o il rumore strumentale, mentre le differenze tra bande tendono ad attenuare tali effetti, evidenziando meglio le caratteristiche spettrali distintive degli oggetti.

Le differenze fungono inoltre da feature più robuste in presenza di osservazioni rumorose o bande con basso segnale, permettendo al modello di apprendere schemi più informativi.

```
feNuovo=data[['u','g','r','i','z','u-g','g-r','r-i','i-z']]
```

Accuratezza: 0.8693				
Risultato di classificazione:				
	precision	recall	f1-score	support
0	0.91	0.94	0.92	11909
1	0.80	0.76	0.78	4371
2	0.82	0.77	0.79	3720
accuracy			0.87	20000
macro avg	0.84	0.82	0.83	20000
weighted avg	0.87	0.87	0.87	20000

Figura 2: Accuratezza con il nuovo dataset

Come si può vedere in Figura [2], sebbene non sia un miglioramento molto consistente, l'algoritmo migliora leggermente le sue prestazioni.

Per questo motivo anche nei futuri modelli verrà tenuto questo dataset.

4.1.2 Questione del numero dei primi vicini (k)

In questo modello, il parametro k rappresenta il numero di vicini più prossimi che vengono considerati per assegnare una classe a un nuovo punto.

La scelta di questo valore è cruciale per le prestazioni del classificatore, poiché influisce direttamente sul bilanciamento tra rumore e generalizzazione del modello.

Se k è troppo piccolo, ad esempio $k = 1$ o $k = 2$, il modello risulta molto sensibile ai dati di training: una singola osservazione rumorosa o anomala può influenzare fortemente la classificazione.

In questo caso, il modello tende a sovradattarsi (overfitting), descrivendo anche il rumore presente nel dataset.

Al contrario, se k è troppo grande, il modello diventa eccessivamente “smussato”.

Considerando un numero elevato di primi vicini, si includono punti appartenenti a classi diffe-

renti.

Questo porta a una perdita di dettaglio e a una tendenza alla sottostima delle differenze tra le classi, con conseguente underfitting.

Ciò significa che per alti valori del parametro k il sistema tende ad imparare benissimo il dataset, non riuscendo però a distinguere nuovi dati che vengono immessi.

Per questi motivi, la scelta di k ottimale viene solitamente effettuata in modo empirico, ad esempio cercando il valore che massimizza le metriche di valutazione (come l'accuratezza o l'F1-score) sul dataset di validazione.

In questo modo si ottiene un compromesso equilibrato tra la capacità di adattarsi ai dati e quella di generalizzare su nuovi campioni.

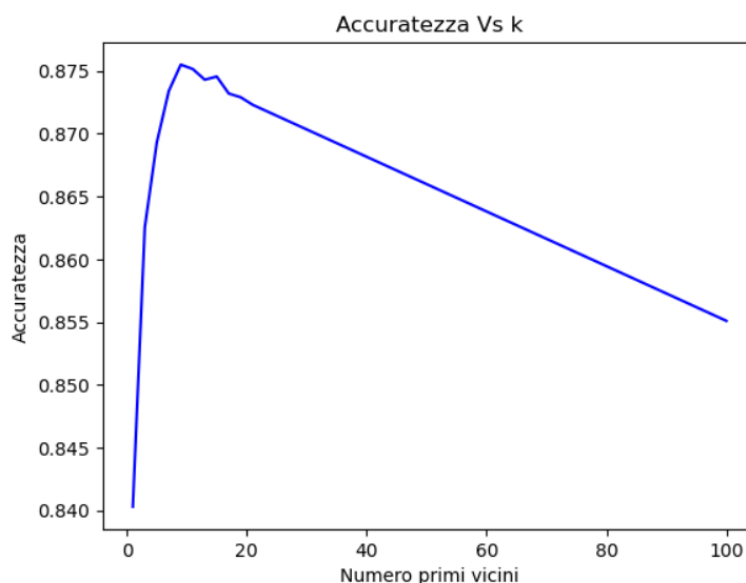


Figura 3: Accuratezza in funzione di k

In Figura [3] si può notare esattamente questa tendenza.

Si può notare come all'aumentare del numero di primi vicini ci sia un picco e successivamente una rapida discesa dell'accuratezza dell'algoritmo.

Siccome nel caso preso in esame il numero di primi vicini per cui si ha il massimo è pari a $k = 11$, verrà usato quello come valore di riferimento.

5 Algoritmo di Bayes

Un altro esempio di algoritmo di ML, è quello di Bayes Naive.

Esso è un algoritmo di apprendimento supervisionato basato sul teorema di Bayes, che consente di stimare la probabilità che un'osservazione appartenga a una determinata classe in base ai valori delle sue caratteristiche.

L'aggettivo naive ("ingenuo") deriva dall'assunzione semplificativa secondo cui tutte le variabili esplicative sono indipendenti tra loro a parità di classe.

Questa ipotesi raramente è verificata nella pratica ma permette di ridurre drasticamente la complessità computazionale.

Formalmente, il teorema di Bayes esprime la probabilità a posteriori come:

$$P(C_i|X) = \frac{P(X|C_i) \cdot P(C_i)}{P(X)} \quad (1)$$

dove:

- $P(C_i|X)$ è la probabilità che un campione con caratteristiche X appartenga alla classe C_i ;
- $P(X|C_i)$ è la verosimiglianza dei dati rispetto alla classe;
- $P(C_i)$ è la probabilità a priori della classe stessa.

Quindi il classificatore assegna "l'osservazione" alla classe con la probabilità a posteriori più elevata.

Il modello Naive Bayes risulta efficace e veloce, soprattutto in problemi con un numero elevato di variabili e dataset di grandi dimensioni, tuttavia la sua semplicità e l'ipotesi di indipendenza (spesso non realistica) lo portano a fare assunzioni non sempre corrette,

Utilizzando quindi il modello di Naive Bayes il codice risulta essere:

```
model=GaussianNB()  
preBey=model.predict(fe_test)
```

Mentre i risultati sono mostrati in Figura [4].

Accuratezza: 0.7042					
Classification Report:					
	precision	recall	f1-score	support	
0	0.76	0.91	0.83	11909	
1	0.54	0.01	0.03	4371	
2	0.57	0.87	0.69	3720	
accuracy			0.70	20000	
macro avg	0.62	0.60	0.51	20000	
weighted avg	0.68	0.70	0.63	20000	

Figura 4: Accuratezza del modello di Bayes

Dalla Figura [4] appare chiaro come l'accuratezza del modello cali drasticamente rispetto al modello precedente del K-nn.

Una delle motivazioni potrebbe essere trovata nelle eccessive semplificazioni che il modello assume, ad esempio che le variabili siano tra loro indipendenti.

Nel caso in esame, le caratteristiche considerate (ad esempio, i parametri fotometrici o le differenze di magnitudine) risultano in parte correlate tra loro, violando l'assunzione di indipendenza condizionale.

Ciò porta a una stima meno accurata delle probabilità a posteriori e, di conseguenza, a una classificazione meno affidabile.

Inoltre, l'algoritmo di Naive Bayes tende a semplificare la distribuzione dei dati all'interno di ciascuna classe (spesso assumendo una distribuzione gaussiana), il che può penalizzare le prestazioni quando le feature non seguono tale distribuzione o presentano andamenti non lineari. Nonostante ciò, il modello rimane utile come baseline di confronto, grazie alla sua velocità, semplicità e capacità di fornire risultati interpretabili con un basso costo computazionale.

6 Random Forest

Il modello Random Forest è anch'esso un algoritmo di machine learning supervisionato. ed è appartenente alla famiglia degli ensemble methods, basato sulla combinazione di numerosi alberi decisionali.

L'idea su cui si basa è quella di costruire una "foresta" di alberi, ciascuno addestrato su un sottoinsieme casuale dei dati e delle variabili, in modo da ridurre la varianza e migliorare la capacità di generalizzazione del modello.

Durante la fase di classificazione, ogni albero esprime una "valutazione" sulla classe di appartenenza dell'osservazione, e la previsione finale viene determinata tramite maggioranza delle decisioni.

Questa strategia consente di ottenere un modello robusto, meno sensibile al rumore e agli outlier rispetto a un singolo albero di decisione.

Il Random Forest è particolarmente efficace in presenza di dataset complessi o non lineari, poiché riesce a catturare relazioni intricate tra le variabili senza necessità di trasformazioni esplicite dei dati.

Tra i suoi principali vantaggi si annoverano l'elevata accuratezza, la resistenza all'overfitting e la possibilità di valutare l'importanza delle feature nel processo decisionale.

Tuttavia, uno degli svantaggi che il modello presenta è il tempo computazionale richiesto, che spesso è molto maggiore rispetto ad altri algoritmi.

Nel caso preso in esame è quindi stato modificato il codice aggiungendo le seguenti linee:

```
model=RandomForestClassifier(n_estimators=100, random_state=26)
model.fit(fe_train, ta_train)
predFor=model.predict(fe_test)
```

Ottenendo i risultati mostrati in Figura [5].

```

Accuratezza: 0.88935

Classification Report:
              precision    recall  f1-score   support

     0           0.92       0.95       0.94       11909
     1           0.85       0.78       0.82        4371
     2           0.82       0.81       0.81        3720

 accuracy                   0.89       20000
 macro avg           0.87       0.85       0.86       20000
 weighted avg        0.89       0.89       0.89       20000

```

Figura 5: Accuratezza Random Forest

Come si nota in Figura [5], i risultati dell' algoritmo mostrano una accuratezza molto elevata, sicuramente molto migliore rispetto ad alcuni metodi già visti come il modello di Bayes (visibile in Figura [4]).

Va tuttavia sottolineato che anche il tempo necessario a questo modello è molto superiore sia del tempo necessario al K-nn che al modello di Naive Bayes.

6.1 Questione del numero di alberi

Come nel modello K-nn, anche il Random Forest dipende fortemente dalla scelta di un parametro fondamentale.

Nel modello Random Forest, questo parametro è il numero di alberi decisionali della foresta (`n_estimators` nel codice).

La sua scelta influisce direttamente sulle prestazioni del classificatore, poiché determina il grado di stabilità e la capacità di generalizzazione del modello.

Un numero troppo basso di alberi può rendere la foresta instabile e sensibile al campionamento casuale: in questo caso, il modello tende a sottostimare la complessità del dataset e può produrre risultati variabili tra un'esecuzione e l'altra.

Al contrario, un numero molto elevato di alberi porta generalmente a un miglioramento delle prestazioni fino a un certo punto, oltre il quale l'accuratezza si stabilizza mentre aumentano i tempi di calcolo e l'utilizzo di memoria.

Nella pratica, la scelta ottimale di `n_estimators` si effettua in modo empirico, valutando le metriche di performance su un set al variare del parametro.

Si ricerca un valore che garantisca un buon equilibrio tra accuratezza e costo computazionale, poiché, grazie al meccanismo di aggregazione, la Random Forest tende a non soffrire di overfitting anche per valori relativamente alti del numero di alberi.

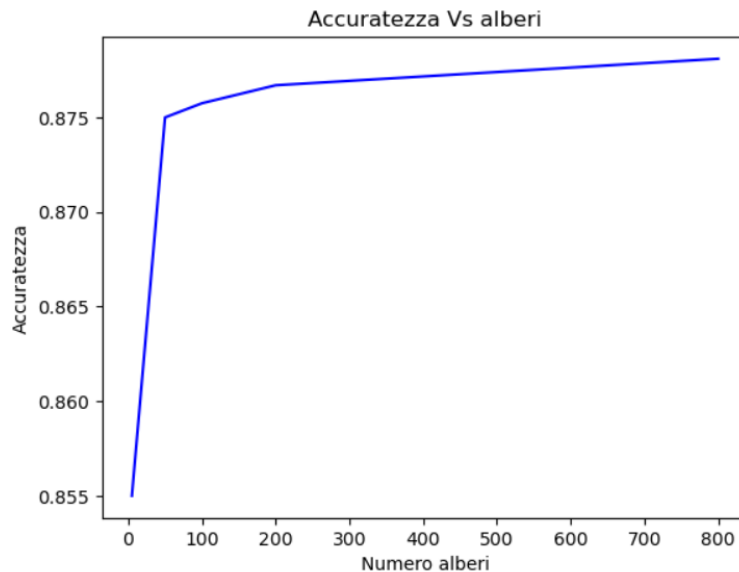


Figura 6: Accuratezza in funzione del numero di alberi

Come si può notare in Figura [6], inizialmente, l'aumento del numero di alberi porta a un miglioramento significativo delle prestazioni, in quanto la foresta diventa più stabile e riduce la varianza del modello.

Tuttavia, oltre una certa soglia il valore dell'accuratezza tende a stabilizzarsi, mostrando una saturazione delle prestazioni. Aggiungere ulteriori alberi non comporta incrementi apprezzabili in termini di accuratezza, ma aumenta il costo computazionale e i tempi di addestramento.

Nel caso preso in esame, si nota come già con pochi alberi l'accuratezza raggiunge valori sufficientemente elevati (quasi il 90%), per questo motivo si è ritenuto sufficiente tenere come valore di riferimento `n_estimators=100`.

7 Confronto tra i tre modelli

Nel complesso, il confronto tra i tre algoritmi di classificazione (k-Nearest Neighbors, Naive Bayes e Random Forest) evidenzia differenze significative in termini di accuratezza, robustezza e complessità computazionale:

- Il modello Naive Bayes si è rivelato il più semplice e veloce da addestrare, ma ha ottenuto prestazioni inferiori, probabilmente a causa dell'ipotesi di indipendenza tra le variabili, non soddisfatta nel dataset analizzato;
- Il k-NN ha mostrato risultati intermedi, pur offrendo una buona accuratezza per valori ottimali di k, risulta sensibile alla scala dei dati e alla presenza di rumore, oltre a richiedere tempi di calcolo elevati in fase di predizione, poiché deve confrontare ogni nuovo campione con l'intero set di training;
- Il modello Random Forest ha invece raggiunto le migliori prestazioni complessive, mostrando un'elevata accuratezza e una buona capacità di generalizzazione. Ciò è dovuto alla combinazione di più alberi decisionali che riduce la varianza e mitiga l'overfitting tipico dei singoli alberi. Tuttavia, questo approccio comporta tempi di addestramento più lunghi e una minore interpretabilità rispetto agli altri modelli.

Nel complesso, la Random Forest rappresenta il miglior compromesso tra affidabilità, robustezza e capacità predittiva, risultando il modello più adatto per il problema considerato.

8 Curva ROC

Una volta appurato che il Random Forest è il modello migliore per il caso preso in esame, può essere utile calcolare la curva ROC,

La curva ROC (Receiver Operating Characteristic) è uno strumento grafico utilizzato per valutare la capacità discriminante di un classificatore binario.

Essa mostra la relazione tra il *True Positive Rate* e il *False Positive Rate* per diversi valori di soglia.

Un modello efficace produce una curva che si avvicina all'angolo superiore sinistro del grafico, mentre un classificatore casuale si dispone lungo la diagonale.

L'area sotto la curva (AUC) fornisce una misura quantitativa delle prestazioni globali: Valori prossimi a 1 indicano un'elevata accuratezza, mentre valori intorno a 0.5 denotano un comportamento completamente aleatorio.

Siccome la curva ROC è un indicatore binario, nel caso preso in esame (un caso di tre classi), la curva è stata calcolata *uno vs gli altri due*.

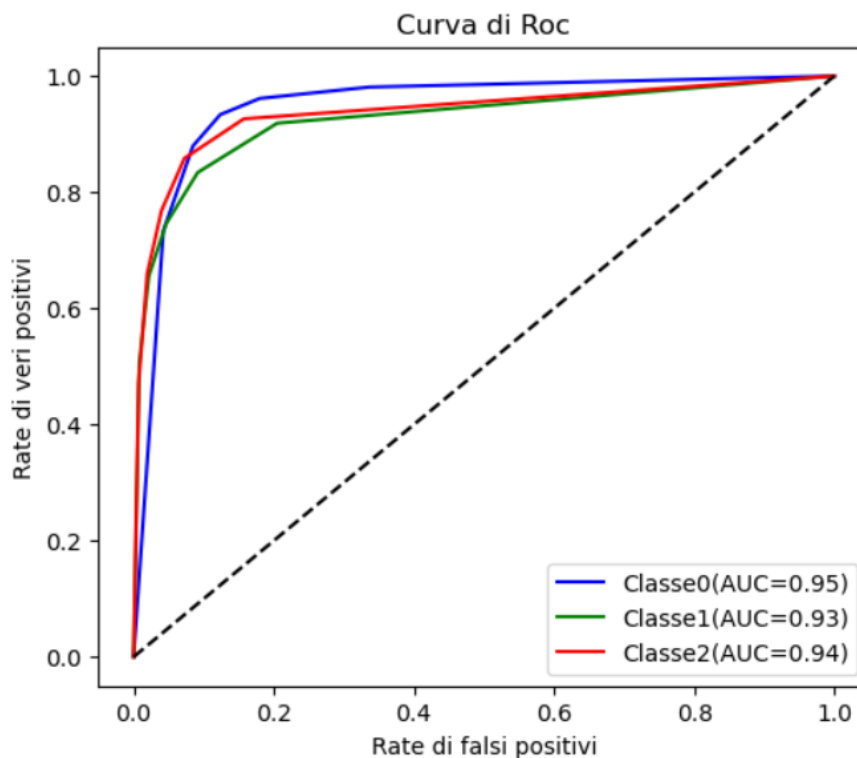


Figura 7: Curva ROC

In Figura [7] si può notare come tutte le curve risultano significativamente al di sopra della diagonale casuale, indicando che il classificatore possiede un'eccellente capacità di distinzione

tra le diverse tipologie di oggetti astronomici.

Si osserva una leggera superiorità nella classificazione delle galassie, mentre le prestazioni per stelle e quasar risultano molto simili e comunque di alto livello.

- Classe 0 (Galassie) = 0.95;
- Classe 1 (Stelle) = 0.93;
- Classe 2 (Quasar) = 0.94.

Complessivamente, la Random Forest mostra una notevole robustezza e affidabilità, confermando la sua efficacia come algoritmo di classificazione in questo contesto.

9 Validazione K-fold

Per valutare la stabilità e la capacità di generalizzazione del modello, è possibile applicare una procedura di validazione incrociata (k-fold cross validation).

In questo approccio, il dataset viene suddiviso in k sottoinsiemi di pari dimensione.

Il modello viene addestrato e testato k volte, utilizzando ogni volta un sottoinsieme diverso per la fase di test e i restanti per l'addestramento.

La media delle metriche ottenute fornisce una stima più robusta delle prestazioni complessive, riducendo la dipendenza dai dati di partizione e migliorando l'affidabilità della valutazione.

Nel contesto di questo lavoro, una validazione di questo tipo permetterebbe di verificare la coerenza dei risultati ottenuti dal modello del Random Forest, assicurando che le alte performance osservate (in termini di accuratezza e AUC) non siano dovute a una specifica divisione del dataset, ma riflettano una reale capacità predittiva.

Si è deciso di utilizzare $k = 5$ fold e le accuratezze ottenute sono risultate pari a $[0.8829, 0.8882, 0.8858, 0.8850, 0.8902]$, con una media complessiva di 0.8864 e una deviazione standard di 0.0026.

Questi valori indicano che il modello presenta prestazioni elevate e molto consistenti su diverse partizioni del dataset, con variazioni minime tra un fold e l'altro.

Ciò conferma la stabilità e l'affidabilità del classificatore Random Forest, che riesce a mantenere un comportamento coerente indipendentemente dalla suddivisione dei dati di addestramento e test.

10 Curva di apprendimento

Una volta sicuri della stabilità e dell'affidabilità del modello, può essere utile studiarlo in funzione del dataset, tramite una curva di apprendimento.

La curva di apprendimento (learning curve) è uno strumento fondamentale per analizzare il comportamento di un modello di machine learning al variare della quantità di dati utilizzati per l'addestramento.

Essa mostra come varia la prestazione del modello al crescere della quantità di dati utilizzati per l'addestramento.

In particolare, sull'asse orizzontale è riportata la dimensione del set di training, mentre sull'asse verticale è rappresentata una metrica di valutazione — in questo caso, l'accuratezza.

Si tracciano quindi due curve distinte: una relativa ai dati di addestramento e una ai dati di validazione (o di test).

Se la curva di training mostra valori molto elevati mentre quella di validazione resta significativamente più bassa, il modello è affetto da overfitting, cioè si adatta troppo ai dati di training e generalizza male.

Al contrario, se entrambe le curve si mantengono su valori bassi, si parla di underfitting, segno che il modello è troppo semplice o che i dati non contengono informazioni sufficienti.

L'analisi della curva di apprendimento consente quindi di diagnosticare eventuali problemi di complessità del modello e di valutare se l'aggiunta di nuovi dati o la modifica dei parametri di addestramento possa migliorare le prestazioni.

In generale, la convergenza delle due curve su valori elevati e vicini tra loro indica che il modello ha raggiunto un buon equilibrio tra accuratezza e capacità di generalizzazione.

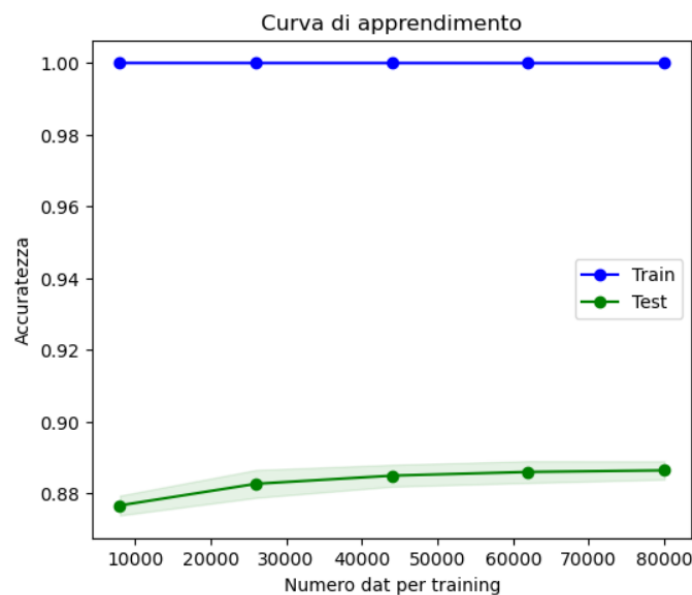


Figura 8: Curva di apprendimento

in Figura [8] si può notare il grafico relativo a tale curva.

Si osserva che l'accuratezza sul set di training (linea blu) rimane praticamente costante e molto vicina al 100%, mentre quella sul set di test (linea verde) cresce leggermente all'aumentare dei dati, stabilizzandosi intorno a un valore di circa 0.89.

Invece l'alone colorato rappresenta la deviazione standard, la quale rimane su valori significativamente bassi.

Questo comportamento indica che il modello ha imparato molto bene i dati di addestramento, ma mostra un leggero overfitting, poiché le prestazioni sul training set sono perfette mentre quelle sul test set restano inferiori.

Tuttavia, la stabilità della curva verde suggerisce che il modello ha raggiunto una buona capacità

di generalizzazione, quindi aggiungere ulteriori dati non migliorerebbe in modo significativo le prestazioni.

11 Reti neurali

11.1 Introduzione alle Reti Neurali

Sebbene il modello del Random Forest abbia mostrato performance più che soddisfacenti, nonchè migliori tra i modelli testati, l'implementazione di una rete neurale consente di verificare se un approccio basato sul deep learning può confermare o migliorare i risultati, fornendo un confronto completo.

Le reti neurali artificiali (*ANN*) sono modelli di apprendimento automatico ispirati al cervello umano ed al suo funzionamento.

Esse sono costituite da nodi (neuroni artificiali chiamati *perceptroni*) organizzati in strati, collegati da pesi numerici che vengono ottimizzati durante la fase di addestramento.

Ogni neurone calcola una combinazione pesata degli ingressi e applica una funzione di attivazione non lineare, permettendo così di modellare relazioni complesse tra le variabili di input e le classi di output.

In questo elaborato viene utilizzata una rete neurale feedforward (Multi-Layer Perceptron, *MLP*) con due strati nascosti e funzione di attivazione *ReLU*.

L'algoritmo di addestramento si basa sul metodo della retropropagazione dell'errore e sull'ottimizzatore *Adam*.

Il modello viene applicato alla classificazione di oggetti astronomici, allo scopo di valutare se una struttura non lineare possa migliorare la capacità di generalizzazione rispetto agli altri algoritmi testati, i quali avevano un livello di complessità molto inferiore.

11.2 Funzionamento delle Reti Neurali

Il funzionamento delle reti neurali è suddiviso in tre strati:

- Input layer: riceve le features dal database, ovvero i dati grezzi che poi saranno analizzati;
- Hidden layer: uno o più strati nascosti ed intermedi tra input e output, dove emergono relazioni non conosciute a priori tra le features;
- Output: mostra la previsione finale.

Ogni singolo neurone, quindi, riceve i dati in input, li processa come una sinapsi cerebrale e restituisce dei valori in output.

La funzione che processa questi valori è la seguente:

$$y = f(\sum w_i \cdot x_i + b) \quad (2)$$

In Eq. (2) si notano i dati in input x_i e un bias b , mentre i valori w_i sono i pesi associati. Essi sanciscono l'importanza di tali input nel processo decisionale, maggiore è il peso w_i , maggiore sarà l'impatto di tale caratteristica.

Questo meccanismo è fondamentale per trovare pattern nascosti, il motivo è che la rete neurale, cercando il migliore set di pesi, finisce con il dare maggiore importanza ad alcuni dati anziché ad altri, rilevando connessioni importanti tra le features e scartando le connessioni non utili (assegnando alla feature un valore prossimo allo zero).

Il ruolo dei bias b è fondamentale in questo senso perché permette alla funzione di essere traslata in alto o in basso lungo la retta che sancisce l'attivazione, senza dovere necessariamente passare per il punto 0.

Difatti, nel caso di un valore $b > 0$ il neurone tenderà ad attivarsi più frequentemente, mentre nel caso di un valore $b < 0$ il neurone si attiverà più raramente.

In generale questo permette all'algoritmo di poter modellare meglio i dati e di essere molto più flessibile.

Tale funzione di attivazione f , quindi, aggiunge non linearità al sistema, introducendo un grado di complessità maggiore rispetto agli algoritmi visti in precedenza, proprio perché i neuroni non *devono* attivarsi, bensì *possono* attivarsi.

Una volta che l'algoritmo ha effettuato la sua predizione, questa viene confrontata con i dati reali, proprio come succedeva nei precedenti modelli supervisionati,

In questo modo si può calcolare una *loss function* (funzione di perdita) che misura quanto il valore predetto si allontani dalla realtà dei dati.

Ovviamente, l'algoritmo ha prestazioni migliori, tanto più riesce a minimizzare la loss function. Proprio per questo, durante la fase di back propagation, il modello cerca i pesi che maggiormente hanno contribuito a fare crescere l'errore e li modifica al fine di ridurre maggiormente la loss function.

In genere il peso delle singole variabili viene aumentato nel caso in cui questo riduca il valore della loss function, e viene abbassato nel caso in cui esso lo aumenti.

Questo procedimento viene ripetuto per numerosi cicli (dette *epoche*), al fine di raggiungere dei valori ottimali.

11.2.1 Scelta della funzione di attivazione

Nella rete neurale utilizzata, la funzione di attivazione scelta per gli strati nascosti è la *ReLU* (Rectified Linear Unit).

ReLU è definita come:

$$f(x) = \max(0, x)$$

essa ha il compito di introdurre non linearità nel modello, permettendo ai neuroni di attivarsi solamente quando il valore di input supera lo zero.

Ciò rende la rete capace di apprendere pattern complessi e non lineari, impossibili da modellare con semplici combinazioni lineari.

ReLU presenta anche vantaggi computazionali, difatti è estremamente semplice da calcolare e accelera notevolmente l'addestramento rispetto ad attivazioni più lente come *sigmoid* o *tanh*.

Lo si può notare nel codice in `activation='relu'`.

11.2.2 Scelta della funzione di ottimizzazione

Sebbene l'obiettivo dell'ottimizzatore non cambi, la sua scelta non è univoca.

L'ottimizzatore più semplice è la stochastic gradient descent (SGD), che modifica i pesi nella direzione opposta al gradiente della loss. Tuttavia, esso può risultare lento, instabile e molto sensibile alla scelta del learning rate.

Per questo motivo si è preferito utilizzare un altro attivatore: Adam (*Adaptive Moment Estimation*).

Nel codice lo si può notare in `solver='adam'`.

Adam combina due idee fondamentali: il momentum, che tiene conto della direzione dei gradienti passati per rendere l'aggiornamento più stabile e veloce, e un learning rate adattivo, che assegna a ciascun peso un passo di aggiornamento personalizzato in base alla sua variabilità.

Questa combinazione consente ad Adam di adattarsi automaticamente alla geometria del problema, accelerare la convergenza e ridurre le oscillazioni nei punti critici della funzione di perdita.

Adam è stato scelto anche per la sua capacità di funzionare bene anche senza un tuning eccessivamente fine dei parametri.

11.3 Algoritmo

Nel caso preso in esame, il set di dati viene inizialmente diviso in set di training e set di test, proprio come nei modelli precedenti.

```
X_train,X_test,y_train,y_test=train_test_split(feNuovo,
taNuovo, test_size=0.2, random_state=26) 2
```

Viene successivamente definita la struttura dellarete neurale:

```
mlp=MLPClassifier(hidden_layer_sizes=(100, 50),
activation='relu', solver='adam', max_iter=500, random_state=26)2.
```

Nel caso in esame, il primo strato contiene 100 neuroni, mentre il secondo ne contiene 50. (`hidden_layer_sizes=(100, 50)`).

La scelta è stata effettuata tenendo conto anche del tempo computazionale richiesto, così da non inficiare troppo sull'efficienza dell'algoritmo.

Il numero di epoche è invece stato scelto pari a 500 (`max_iter=500`).

²La line di codice viene scritta andando a capo unicamente per motivazioni di spazio.

Seguono infine, la parte di addestramento e la parte di predizione:

```
mlp.fit(X_train, y_train)
y_pred_mlp=mlp.predict(X_test)
```

Accuratezza: 0.881

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.94	0.93	11909
1	0.83	0.77	0.80	4371
2	0.81	0.80	0.80	3720
accuracy			0.88	20000
macro avg	0.85	0.84	0.85	20000
weighted avg	0.88	0.88	0.88	20000

Figura 9: Accuratezza rete neurale.

In Figura [9] si può notare l'accuratezza della rete neurale.

Tale accuratezza assume certamente un valore molto elevato, circa 88%, tuttavia non si discosta molto dai valori trovati in precedenza analizzati in Paragrafo [6].

11.4 Matrice di confusione

Al fine di mostrare i risultati con maggiore chiarezza, può essere utile utilizzare una matrice di confusione.

Essa indica il modo in cui l'algoritmo sbaglia e dove sbaglia maggiormente.

Nel caso preso in esame le righe rappresentano le classi reali, ovvero i valori corretti del dataset, mentre le colonne rappresentano le classi predette dal modello.

Ogni cella contiene il numero di elementi che appartengono a una certa classe reale e che sono stati predetti come un'altra classe.

Nello specifico la diagonale principale, che parte in alto a sinistra per giungere in basso a destra, contiene i casi in cui il modello ha classificato correttamente, viceversa, le celle fuori dalla diagonale mostrano gli errori e quali classi il modello tende a confondere.

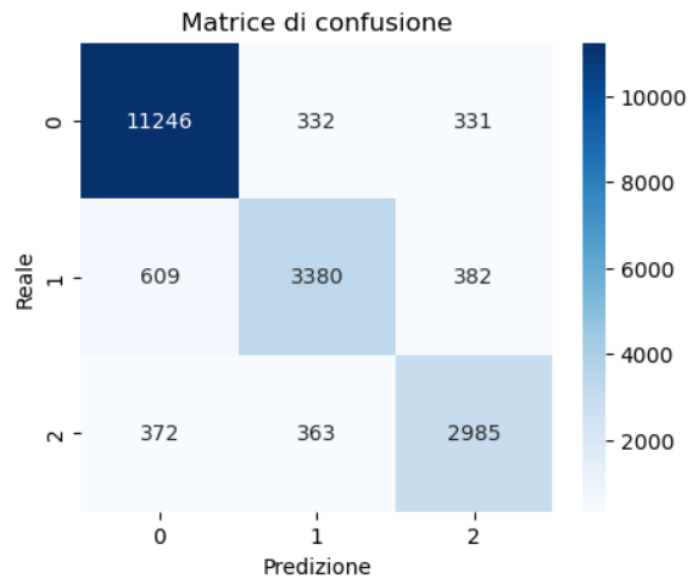


Figura 10: Matrice di confusione.

Come si può notare in Figura [10], la matrice di confusione mostra che il modello classifica con estrema accuratezza gli elementi della classe 0 (ovvero le Galassie), mentre riscontra maggiori difficoltà nel distinguere correttamente le classi 1 e 2 (ovvero Stelle e Quasar).

In particolare, la classe delle stelle è quella maggiormente confusa: 609 elementi vengono classificati come galassie e 372 come quasar, risultando quindi la classe più problematica per il modello.

La classe delle quasar presenta invece prestazioni intermedie, con 2985 predizioni corrette.

Nel complesso, la matrice evidenzia una buona capacità del modello per alcune classi, ma anche squilibri nelle performance, suggerendo che la rete neurale non riesce a separare perfettamente tutte le categorie.

12 Conclusione

Nonostante la rete neurale rappresenti un modello nettamente più flessibile e concettualmente più complesso, nel caso in esame il Random Forest ha ottenuto prestazioni comparabili o addirittura superiori.

Questo risultato è giustificabile perchè il modello del Random Forest tende a performare molto bene su dataset tabellari di dimensioni ridotte o moderate, mentre le reti neurali richiedono generalmente dataset più ampi e un tuning più accurato per superare tali modelli.

Si sottolinea, inoltre, che la rete neurale ha un costo computazionale nettamente maggiore del modello del Random Forest, rendendo quest'ultimo ancora più vantaggioso.

Si può quindi concludere notando che, oltre ad aver trovato l'algoritmo che meglio si adatta al caso corrente, l'algoritmo riesce a distinguere le tre classi richieste in maniera soddisfacente.

Riferimenti bibliografici

- [1] *Probabilistic Machine Learning An Introduction*. (2022) Kevin P. Murphy
- [2] *Pattern Recognition and Machine Learning*. (2006) Christopher Bishop
- [3] *Pattern Classification*. (1973) Richard O. Duda, Peter E. Hart, David G. Stork
- [4] STACK OVERFLOW web site